Quantstamp

Hemi - Vetro VUSD

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Pegged
Timeline	2026-02-16 through 2026-02-20
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Specification	None
Source Code	• https://github.com/hemilabs/vetro-contracts #b0507cb
Auditors	• Andrei Stefan Auditing Engineer • Ibrahim Abouzied Auditing Engineer • Roman R Senior Auditing Engineer

Documentation quality	Medium
Test quality	High
Total Findings	8 Fixed: 4 Acknowledged: 3 Mitigated: 1
High severity findings	1 Fixed: 1
Medium severity findings	3 Fixed: 2 Acknowledged: 1
Low severity findings	3 Acknowledged: 2 Mitigated: 1
Undetermined severity findings	0
Informational findings	1 Fixed: 1

Summary of Findings

The Vetro Protocol is a fully-collateralized pegged token issuance system that allows users to deposit collateral assets (e.g., USDC, USDT) to mint pegged tokens (VUSD, VETH, VBTC) at oracle-determined rates. Deposited collateral is deployed into external yield-bearing vaults, and the generated yield is distributed to stakers who lock their pegged tokens in an ERC-4626 `StakingVault`. The protocol supports algorithmic market operations `(AMO)` for liquidity provisioning and implements tiered withdrawal delay mechanisms to protect against bank-run scenarios.

The Audit highlights several security vulnerabilities and operational concerns regarding its smart contracts, particularly in the areas of withdrawal mechanisms, oracle dependencies, and privileged role permissions. Key medium severity findings include issues with the `StakingVault` that allow users to compute asset conversions without accounting for accrued yield, leading to potential value leakage. Additionally, a design flaw in the `Gateway`'s `requestRedeem` function inadvertently locks matured tokens, extending their claim period unexpectedly. There are also implications related to the unchecked burning of `PeggedToken`, which can desynchronize accounting for asset-backed supply, increasing risks for users and stakers.

Recommendations focus on enhancing operational safeguards and improving contract architecture. It is advised to implement additional access control mechanisms to prevent misuse by privileged roles, delegate financial management more transparently, and develop a more dynamic price tolerance system that accounts for different collateral profiles. Importantly, mechanisms should be established to handle depegging events effectively to mitigate bank-run scenarios. To resolve these vulnerabilities before production, the audit suggests rigorous input validation, reworking oracle price validation to accommodate non-stable assets, and enforcing more granular controls over parameter changes to prevent unexpected behaviors in the protocol's functionalities.

Fix Review Update: In the recent updates, vulnerabilities HV-1, HV-2, HV-3, and HV-8 have been successfully fixed while HV-6 has been mitigated with a partial fix. Vulnerabilities HV-4 and HV-5 have been acknowledged by the client with explanations regarding their design choices: HV-4 pertains to the use of stable coins only for collateral, while HV-5 relates to a minimal withdrawal delay, which the client believes does not necessitate a separate withdrawal request process. Additionally, vulnerability HV-7 has also been acknowledged, with the client noting that VUSD will rely on highly stable coins and that monitoring mechanisms will be in place for withdrawal control. Lastly, suggestions S1, S2, S3, S4, S6, and S7 have all been implemented successfully, though suggestion S5 remains acknowledged with an explanation regarding trust in the AMO role and its minting practices.

ID	DESCRIPTION	SEVERITY	STATUS
HV-1	Unpulled Accrued Yield Can Become Permanently Orphaned on Subsequent `distribute()` Calls	High	Fixed
HV-2	`StakingVault.requestRedeem()` & `StakingVault.requestWithdraw()` Computes Share-Asset Conversion Before Pulling Yield	Medium	Fixed
HV-3	AMO Supply Invariant Can Break if AMO-Minted Tokens Are Burned Directly	Medium	Fixed
HV-4	Oracle Price Tolerance Assumes Pegged Assets Are $1	Medium	Acknowledged
HV-5	`Gateway.RequestRedeem()` Re-Locks Already-Matured Claimable Tokens	Low	Acknowledged
HV-6	Unbound Fee Parameters May Lead to Unexpectedly Extreme Values	Low	Mitigated
HV-7	Depeg Scenario Creates a Bank-Run Dynamic	Low	Acknowledged
HV-8	Boolean Toggle Functions Can Counteract Each Other	Informational	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
    1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
    2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
    3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
    1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
    2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.

4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: https://github.com/hemilabs/vetro-contracts(b0507cb46a4b87cde9087e2954b4575ca24ce7c7)

Files: src/*.sol
Repo: `https://github.com/hemilabs/vetro-contracts`

Included Paths: `src/`

Files Excluded

Repo: https://github.com/hemilabs/vetro-contracts(b0507cb46a4b87cde9087e2954b4575ca24ce7c7)

Files: ./test/, *./lib/*

Operational Considerations

1. **Oracle Price Tolerance as Circuit Breaker**: The `Treasury` enforces a global `priceTolerance` (default 1%) that validates all oracle prices are within +-N% of the unit price (1.0). This acts as a circuit breaker: if any collateral token depegs beyond tolerance, ALL deposits and withdrawals for that token are halted. TThe oracle is expected to report relative peg prices (e.g., USDC/USD ≈ 1.0, WETH/ETH ≈ 1.0), not absolute prices. During extended depeg events, the `MAINTAINER_ROLE` must widen tolerance or wait for recovery. Users with pending redeem requests cannot claim during such events.
2. **Isolated Deployment Per Peg Target**: Each pegged asset (VUSD, VETH, VBTC) is deployed as a completely independent stack: `PeggedToken` → `Gateway` → `Treasury` → `StakingVault` → `YieldDistributor`. There is zero shared state between deployments. A collateral depeg in one deployment cannot affect another. Within a single `Treasury`, multiple collateral tokens are supported via whitelist (e.g., USDC + USDT + DAI for VUSD). The `reserve()` function aggregates across all whitelisted tokens.
3. **AMO Supply Accounting**: The `Gateway` tracks `amoSupply` separately from user-minted supply. The invariant `amoSupply <= totalSupply` must always hold for `maxMint()` and `_excessTokens()` to function correctly. AMO tokens are minted without backing collateral and tracked so that `_backedSupply = totalSupply − amoSupply` represents the collateral-backed portion. This separation allows the `UMM_ROLE` to harvest only genuine excess (yield profits) from the `Treasury`.
4. **Dual Withdrawal Delay Systems**: The protocol implements two independent delay mechanisms: (1) `Gateway` withdrawal delay for `PeggedToken` redemptions, with a single request slot per user that accumulates and resets the timer on each `requestRedeem()` call, and (2) `StakingVault` cooldown for vault share redemptions, supporting multiple concurrent requests per user with independent timers. Both systems have instant bypass whitelists for trusted addresses.
5. **Yield Distribution Pattern**: The `YieldDistributor` linearly drips yield to the `StakingVault` over a configurable period (default 7 days) to prevent sandwich attacks. The `StakingVault` calls `_pullYield()` before every state-changing operation (`deposit`, `mint`, `redeem`, `withdraw`, request, cancel, claim) to ensure share prices reflect accrued yield. When `totalSupply` is 0, yield pulling is skipped to prevent orphaned yield from unfairly benefiting future depositors.
6. **PeggedToken Blacklist Impact**: The blacklist in `PeggedToken` prevents all transfers to/from blacklisted addresses, including minting and burning. If the `Gateway` address were blacklisted, all protocol operations would halt. If a user with a pending redeem request in the `Gateway` is blacklisted, they cannot cancel their request (the `safeTransfer` back to them would revert), effectively locking their tokens in the `Gateway`.
7. **Treasury Migration Limitations**: The `migrate` function transfers raw token balances and vault share balances to a new treasury but does NOT transfer whitelist configuration, oracle settings, role assignments, or other state. The new treasury must be fully configured independently. Migration also requires the new treasury to have the same `PEGGED_TOKEN` reference.
8. **Canceling a StakingVault Withdrawal May Return Fewer Shares**: `StakingVault.cancelWithdraw()` is callable even after the cooldown has matured. Shares returned are calculated at cancellation time via `previewDeposit(_assets)` after pulling yield. Since the share price increases over time due to yield distribution, a user who cancels will generally receive fewer shares than they originally burned. Users should not assume share-for-share equivalence when canceling.
9. **Redemption Token is Not Linked to Deposit Token**: In `Gateway.redeem()` / `Gateway.withdraw()`, the user selects the `tokenOut_` at redemption time. There is no on-chain linkage between a user's original deposit token and the token they redeem. Users should not assume they will receive the same collateral they originally deposited.
10. **Excess Harvest is Constrained by Per-Token Liquidity**: `Treasury.harvest()` attempts to withdraw excess for a specific token, but payment is limited to that token's available balance (raw + vault). Even if the protocol has significant excess reserve value overall, there may not be enough of a particular token to fully realize that excess.
11. **VaultRewards External Contract Interaction**: The `StakingVault` calls `IVaultRewards.updateReward()` on every token transfer (mint, burn, transfer) via the `_update()` hook. This external contract and its interaction are out of scope for this audit. A misconfigured or malicious `vaultRewards` address could revert on `updateReward()`, blocking all sVUSD transfers.
12. **Same-Block Deposit and Redeem with Matured Request**: If a user has a matured (unlocked) redeem request in the `Gateway`, they can deposit collateral and redeem within the same transaction or block, up to the unlocked amount. This is expected behavior but worth noting for monitoring purposes.
13. **Yield Distribution is Manually Triggered**: The yield distribution via the `YieldDistributor` is manually invoked by the `DISTRIBUTOR_ROLE` and depends on the protocol operators performing it regularly and with sensible amounts. There is no on-chain enforcement of distribution frequency or amount correctness.

14. **Fee/Peg Band Configuration:** We assume mint/redeem fees are set appropriately relative to the peg bands. If fees are too low, the protocol may subsidize small depegs that are within the peg band on behalf of the end user, eating into the protocol reserves over the long-term.

Key Actors And Their Capabilities

1. **Users**: Can deposit whitelisted collateral tokens (e.g., USDC, USDT) via the `Gateway` to receive `PeggedToken` (e.g., VUSD) minted 1:1 relative to oracle price. Users can mint by specifying desired output or deposit by specifying input amount. Users can redeem `PeggedToken` for collateral via instant redeem (if whitelisted or delay disabled) or through the delayed withdrawal mechanism by calling `requestRedeem` and waiting the configured delay period before claiming. Users can cancel pending redeem requests to recover locked `PeggedToken`. Can stake `PeggedToken` in the `StakingVault` to earn yield, receiving `ERC4626` vault shares. Can request cooldown withdrawals from `StakingVault` with multiple concurrent requests tracked independently, cancel cooldown requests (re-minting shares at current exchange rate), and batch-claim matured requests.
2. **DEFAULT_ADMIN_ROLE (Treasury Admin)**: Controls strategic protocol configuration with a 3-day admin transfer delay via `AccessControlDefaultAdminRules`. Can add and remove whitelisted collateral tokens with their corresponding yield vaults, oracle feeds, and stale periods. Can migrate all treasury assets to a new treasury contract. Can sweep non-reserved tokens from the treasury. Can update the Gateway's mint limit and AMO mint limit. Can grant and revoke all other roles `(KEEPER_ROLE, MAINTAINER_ROLE, UMM_ROLE)`. Effectively controls the `Gateway` since its delegates all access control to the Treasury's role system.
3. **MAINTAINER_ROLE**: Handles protocol parameter tuning. Can update oracle addresses and stale periods for whitelisted tokens. Can update the global price tolerance parameter. Can update the swapper contract address. Can update mint and redeem fee percentages (0-99.99%). Can toggle the Gateway's withdrawal delay feature on/off. Can update the withdrawal delay period. Can add or remove addresses from the Gateway's instant redeem whitelist.
4. **KEEPER_ROLE**: Manages day-to-day treasury operations. Can push (deposit) idle collateral from treasury into yield vaults. Can pull (withdraw) collateral from yield vaults back to treasury. Can toggle deposit and withdrawal activity per-token to pause/unpause operations. Can execute token swaps through the configured swapper contract for non-reserved tokens. Keeper is allowed to push/pull even when deposit/withdraw is paused for the token.
5. **UMM_ROLE (Algorithmic Market Operations related role)**: Can mint unbacked `PeggedToken` via `mintToAMO` to any receiver address within the AMO mint limit for liquidity provisioning. Can burn `PeggedToken` via `burnFromAMO` from their own balance to reduce AMO supply tracking. Can harvest excess reserve from the treasury (reserve above backed supply).
6. **PeggedToken Owner**: Separate from `Treasury` admin. Controls the `PeggedToken` contract directly via `Ownable2Step`. Can update the gateway and treasury addresses on the `PeggedToken`. Can add or remove addresses from the `PeggedToken` blacklist, preventing all transfers to/from blacklisted addresses.
7. **StakingVault Owner**: Controls the `StakingVault` via `Ownable2StepUpgradeable`. Can update cooldown duration (1-30 days), enable/disable cooldown mechanism, manage instant withdraw whitelist, and set the `YieldDistributor` and `VaultRewards` contract addresses.
8. **YieldDistributor Admin**: Controls the YieldDistributor via `AccessControlDefaultAdminRulesUpgradeable` with 3-day transfer delay. Can update yield distribution duration (minimum 1 day). Can rescue accidentally sent tokens (except the distribution asset). Manages the `DISTRIBUTOR_ROLE`.
9. **DISTRIBUTOR_ROLE**: Can call `distribute` on YieldDistributor to add new yield for linear distribution to the StakingVault. Remaining undistributed yield is combined with new amounts and the distribution period resets.
10. **Privileged Role Attack Vectors**: Several multi-step attack paths exist if privileged roles are compromised. A compromised `MAINTAINER_ROLE` could replace a token's oracle via `Treasury.updateOracle()` with a malicious feed, manipulating all mint/redeem price calculations across the `Gateway`. Similarly, `Treasury.updateSwapper()` allows setting an arbitrary swapper address — since `Treasury.swap()` approves and calls into this address, a malicious swapper could drain approved funds. A compromised `DEFAULT_ADMIN_ROLE` could call `removeFromWhitelist()` on a collateral token and then immediately `sweep()` the now-unprotected collateral to an arbitrary receiver.

Findings

HV-1

Unpulled Accrued Yield Can Become Permanently Orphaned on Subsequent `distribute()` Calls

High Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `b725879`.

File(s) affected: `src/YieldDistributor.sol`

Description: The `YieldDistributor.distribute()` function resets `lastUpdateTime` to the current timestamp and recalculates `rewardRate` using only the remaining undistributed portion of the current schedule plus the new distribution amount.

If yield has already accrued since the previous `lastUpdateTime` but has not yet been pulled via `pullYield()`, that accrued portion is neither transferred nor carried forward into the new schedule.

This results in:

- Accrued but unpulled yield is excluded from the new emission schedule.
- Those tokens remain in the contract balance.
- They are no longer accounted for in `pendingYield()`.
- They cannot be withdrawn via `pullYield()`.
- They cannot be rescued via `rescueTokens()` because it can't transfer the asset token.

- This leads to permanent token orphaning and breaks the invariant that: total tokens distributed == total tokens eventually pullable

The issue occurs whenever:

- A distribution is active.
- Time passes (>= 1 block) without a `pullYield()`.
- A new `distribute()` is called before a pull.

Example:

0. Assume: `yieldDuration = DEFAULT_YIELD_DURATION = 7 days`
1. At day 0: `distribute(70)`
    - `_rewardRate = 10 tokens/day`
    - `periodFinish = day 7`
    - `lastUpdateTime = day 0`
    - `Contract balance = 70`
2. Time passes without pull
3. At day 3:
    - `Accrued yield = 3 × 10 = 30`
    - `Remaining scheduled = 4 × 10 = 40`
    - `Contract balance still = 70`
4. New distribution before pull at Day 3: `distribute(70)`
    - `_remaining = 40`
    - `_newTotal = _remaining + 70 = 110`
    - It sets a new 7-day schedule for 110 tokens and resets: `lastUpdateTime = Day 3`
    - However, the contract balance is: `old 70 + new 70 = 140`

There are now: `140 − 110 = 30 tokens` that are not included in the emission schedule. These 30 tokens are permanently stuck and `pendingYield()` will only account for 110 tokens. After the schedule completes, `pendingYield()` becomes zero. The 30 tokens remain in the contract balance indefinitely.

Recommendation: Before recalculating the new reward schedule in `distribute()`, the contract must properly account for all yield accrued since the previous `lastUpdateTime`, not only the remaining future portion.

We recommend one of the following:

1. Compute the total undistributed balance based on the previous schedule using `(periodFinish − lastUpdateTime) * rewardRate` instead of using `(periodFinish − now) * rewardRate`. This ensures accrued but unpulled yield remains part of the new emission schedule.
2. Consider pushing accrued but unpulled funds to the vault.
3. Require `pullYield()` to be called before another `distribute()` can proceed.

HV-2

`StakingVault.requestRedeem()` & `StakingVault.requestWithdraw()` Computes Share-Asset Conversion Before Pulling Yield

Medium · Fixed

Update

Marked as "Fixed" by the client.

Addressed in: `88ef385`.

File(s) affected: `src/StakingVault.sol`

Description: In `StakingVault._requestRedeem()` and `_requestWithdraw()`, the share-to-asset (or asset-to-share) conversion is computed via `previewRedeem` / `previewWithdraw` BEFORE `_pullYield()` is called inside `_createRequest()`. This ordering means the conversion uses stale `totalAssets` that doesn't include pending yield from the YieldDistributor. Inside `_createRequest` at L#437, `_pullYield()` is called which transfers accrued yield into the vault, increasing `totalAssets()`. However, the `assets_` value was already locked in at the pre-yield rate.

For `requestRedeem`: The user locks fewer assets in cooldown than their shares are worth post-yield. The unclaimed yield value leaks to remaining stakers.

For `requestWithdraw`: The user burns more shares than necessary for the requested asset amount, since the pre-yield share price is lower. The excess share value leaks to remaining stakers.

Example: If 50 VUSD of yield is pending and a user with 100 shares (worth 105 assets post-yield) calls `requestRedeem(100)`, they lock only 100 assets instead of 105, losing 5 VUSD in value.

This is inconsistent with the instant withdrawal path where `deposit`, `mint`, `redeem`, and `withdraw` all call `_pullYield()` BEFORE the OZ ERC4626 base computes conversions. The `cancelWithdraw` function also correctly calls `_pullYield()` before `previewDeposit`.

Exploit Scenario:

```
function test_requestRedeem_shouldIncludePendingYield() public {
      // alice deposits 100 VUSD, gets 100 shares
      vm.startPrank(alice);
```

```
    vusd.approve(address(vault), 100 * UNIT);
    vault.deposit(100 * UNIT, alice);
    vm.stopPrank();

    // distribute 10 VUSD yield
    deal(address(vusd), owner, 10 * UNIT);
    vm.startPrank(owner);
    vusd.approve(address(yieldDistributor), 10 * UNIT);
    yieldDistributor.distribute(10 * UNIT);
    vm.stopPrank();

    // time skip
    vm.warp(block.timestamp + 7 days);

    // alice requests redeem of ALL her shares
    vm.prank(alice);
    (, uint256 lockedAssets) = vault.requestRedeem(100 * UNIT, alice);

    // expected: 110 VUSD (100 original + 10 yield)
    // actual (with bug): 100 VUSD (yield not included in preview)
    assertEq(lockedAssets, 110 * UNIT); // THIS WOULD FAIL
}
```

Recommendation: Call `_pullYield()` before the preview calculation in both functions:

```
function _requestRedeem(uint256 shares_, address owner_) internal returns (uint256 requestId_, uint256
assets_) {
    _pullYield();
    assets_ = previewRedeem(shares_);
    requestId_ = _createRequest(assets_, shares_, owner_);
}

function _requestWithdraw(uint256 assets_, address owner_) internal returns (uint256 requestId_, uint256
shares_) {
    _pullYield();
    shares_ = previewWithdraw(assets_);
    requestId_ = _createRequest(assets_, shares_, owner_);
}
```

HV-3

AMO Supply Invariant Can Break if AMO-Minted Tokens Are Burned Directly

Medium Fixed

Update

Marked as "Fixed" by the client.

Addressed in: `9e0fb84` .

File(s) affected: `src/Gateway.sol` , `src/Treasury.sol`

Description: The Gateway maintains an `amoSupply` counter that tracks unbacked PeggedToken minted via `mintToAMO` . The invariant `amoSupply <= totalSupply` is assumed throughout the codebase. However, PeggedToken inherits `ERC20Burnable` , which gives any token holder the ability to call `burn()` or `burnFrom()` to destroy their tokens. If tokens that were originally minted via AMO are burned through these public functions instead of through `burnFromAMO()` , `totalSupply` decreases but `amoSupply` does not.

```
// Gateway.sol:415 (maxMint)
uint256 _userSupply = _totalSupply - _amoSupply;  // underflows if totalSupply < amoSupply

// Treasury.sol:454 (_excessTokens)
uint256 _backedSupply = _totalSupply - _amoSupply;  // same underflow risk
```

The `maxMint()` function reverts, preventing any new user deposits/mints. The `_excessTokens()` function reverts, preventing `harvest()` from extracting yield. These remain bricked until UMM acquires and burns tokens via `burnFromAMO()` to reduce `amoSupply` below `totalSupply` .

Exploit Scenario:

1. Users deposit and mint 1000 VUSD. `totalSupply=1000, amoSupply=0`
2. UMM mints 500 VUSD via AMO to receiver. `totalSupply=1500, amoSupply=500`
3. Users redeem all 1000 VUSD. `totalSupply=500, amoSupply=500`
4. AMO receiver burns 1 VUSD directly via `burn()`. `totalSupply=499, amoSupply=500`
5. `_totalSupply - _amoSupply` underflows → all calls to `maxMint()` and `harvest()` revert

Recommendation:

- Ensure burns that should affect AMO accounting must route through `Gateway`.
- Remove/disable public self-burn (drop `ERC20Burnable` or override `burn()` to revert for non-`gateway`).
- Restrict burning to `burnFrom()` with `msg.sender == gateway`.
- If self-burn is desired, do not allow a holder of the `UMM_ROLE` to call the burn functions.

HV-4 Oracle Price Tolerance Assumes Pegged Assets Are $1

Medium | Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> Each pegged token's oracle returns a price denominated in its corresponding base collateral, not in USD — VUSD uses a USD feed, VBTC uses a WBTC feed, and VETH uses a WETH feed. Since each oracle quotes the pegged token against its backing asset, the price naturally stays near 1.0, and the tolerance check remains valid. Furthermore, only collateral tokens that are closely pegged to the base asset will be whitelisted (e.g., no plan to accept stETH as collateral for VETH). This ensures the oracle price consistently falls within the tolerance band for all supported collateral types.

File(s) affected: `src/Treasury.sol`

Description: The Treasury is responsible for quoting collateral prices through `Treasury.getPrice()` and then using those quotes throughout minting and redemption. The Gateway relies on these prices in `_calculatePeggedTokenOutput()` and `_calculateTokenOutput()` to compute how many `PeggedToken` units to mint or how much collateral to return.

However, `Treasury._getPrice()` derives a `_unitPrice` as `10 ** oracle.decimals()` and then enforces that `_latestPrice` stays within a narrow tolerance band around that `_unitPrice`. This implicitly assumes the oracle answer should be near `1.0` in oracle-decimal units, which only holds for stablecoin/USD feeds.

If the protocol intends to whitelist volatile collateral like ETH or BTC using standard USD price feeds, `_latestPrice` will be far from `_unitPrice` and `Treasury.getPrice()` will revert. Since Gateway's `previewDeposit()`, `previewMint()`, `previewRedeem()`, and `previewWithdraw()` all route through `Treasury.getPrice()`, minting and redemption for those tokens will be unusable, effectively preventing the protocol from supporting non-stable collateral as intended.

Recommendation: Redesign price validation so it does not assume the oracle price must be near 1.0. For example, treat `priceTolerance` as a deviation check against a reference price rather than against `_unitPrice`, or gate the "near 1" tolerance logic behind a configuration flag for stable-asset collateral only.

Ensure the updated approach is compatible with how `Gateway._calculatePeggedTokenOutput()` and `Gateway._calculateTokenOutput()` interpret `(latestPrice, unitPrice)` for both stable and volatile collateral types.

HV-5 `Gateway.RequestRedeem()` Re-Locks Already-Matured Claimable Tokens

Low | Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> This is design choice because withdraw delay will be just few minutes (less than 10 minutes) . For better user experiencing in such small delay creating separate withdraw request is not necessary. If user try to request new redeem , UI can show message/alert about to claim previous withdrawal.

File(s) affected: `src/Gateway.sol`

Description: The Gateway uses a single `RedeemRequest` slot per user. When `requestRedeem()` is called, it unconditionally overwrites the `claimableAt` timestamp to `block.timestamp + withdrawalDelay`, regardless of whether the existing request has already matured and is ready to claim.

```solidity
// Gateway.sol:311-326
function requestRedeem(uint256 peggedTokenAmount_) external nonReentrant {
    if (peggedTokenAmount_ == 0) revert AmountIsZero();

    GatewayStorage storage $ = _getGatewayStorage();
    if (!$.withdrawalDelayEnabled) revert WithdrawalDelayFeatureNotEnabled();

    $.peggedToken.safeTransferFrom(msg.sender, address(this), peggedTokenAmount_);

    RedeemRequest storage _request = $.redeemRequests[msg.sender];
    uint256 _newAmount = _request.amountLocked + peggedTokenAmount_;
    uint256 _newClaimableAt = block.timestamp + $.withdrawalDelay;

    _request.amountLocked = _newAmount;
    _request.claimableAt = _newClaimableAt;  // ← unconditionally resets timer

    emit RedeemRequested(msg.sender, _newAmount, _newClaimableAt);
}
```

There is no guard preventing a user from calling `requestRedeem()` when they already have a fully matured request (i.e., `block.timestamp >= claimableAt`). The function merges the new amount into the existing request and resets the cooldown, effectively re-locking tokens that were already claimable via `redeem()` / `withdraw()`.

Exploit Scenario:
Consider a scenario with a 7-day withdrawal delay:

1. Alice calls `requestRedeem(1000 VUSD)` → `claimableAt = now + 7 days`.
2. 7 days pass. Alice's 1000 VUSD is now claimable.
3. Alice calls `requestRedeem(10 VUSD)` instead of first claiming her matured request.
4. Result: `amountLocked = 1010 VUSD`, `claimableAt = now + 7 days` — the originally claimable 1000 VUSD is re-locked for another full 7 days.

The redemption path in `_handleRedeemOrWithdraw()` properly checks maturity (`block.timestamp >= _request.claimableAt`) before allowing claims, but `requestRedeem()` has no equivalent check to protect matured requests from being overwritten.

Recommendation: Update `Gateway.requestRedeem()` to avoid re-locking already-claimable balances. Follow the `StakingVault` request structure by creating separate redeem requests so new locks do not extend the claim time for previously matured funds

HV-6
Unbound Fee Parameters May Lead to Unexpectedly Extreme Values

Low Mitigated

Update
Marked as "Fixed" by the client.
Addressed in: `a876406`.

Update
During the fix review proces we identifyed that `Gateway.updateMintLimit()` has not been updated.

File(s) affected: `src/Gateway.sol`, `src/Treasury.sol`

Description: Functions `updateMintFee()` and `updateRedeemFee()` allow `MAINTAINER_ROLE` to change the fees to any value between 0% to 99.999% with immediate effect.

While users can protect themselves with the provided slippage protection by specifying expected input/output tokens, this may lead to unexpectedly high or low fee values.

Similarly, the following two cases are impacted:

1. `updateMintLimit()` is impacted and will accept arbitrary values, potentially prohibiting protocol operation if set to prohibitively low values.
2. `Treasury.updateOracle()`: `newStalePeriod_`. Consider adding a sane upper limit to prevent (accidentally) setting to an unreasonably high value like uint256.max and therefore never check for staleness.

Recommendation: Consider adding and enforcing lower and upper bounds that will reflect the expected use cases.

HV-7 Depeg Scenario Creates a Bank-Run Dynamic

Low Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> VUSD is suppose to use highly stable and high cap stable coins like USDC, USDT. Keeper will be monitoring the prices of USDC,USDT and can pause withdraw

File(s) affected: `src/Gateway.sol`

Description: The protocol allows `PeggedToken` holders to redeem into any supported collateral token through the `Gateway.redeem()` / `Gateway.withdraw()` flows, which ultimately call `Treasury.withdraw()` for the chosen `tokenOut`. This design assumes the set of whitelisted collateral tokens remain reasonably stable in value relative to the peg.

If one whitelisted collateral token depegs, `PeggedToken` holders are incentivized to rapidly redeem into the safest remaining collateral rather than the impaired token. Because `Treasury.withdrawable(token)` and the vault balances are finite and redemptions are first-come-first-served, early redeemers can exit into higher-quality collateral while the remaining `PeggedToken` supply becomes increasingly backed by the depegged asset. This creates a bank-run dynamic and concentrates losses on late redeemers, with no on-chain mechanism to socialize losses across all `PeggedToken` holders.

Recommendation: Pause withdrawals and see if the token repegs. If not, use an explicit depeg / insolvency resolution mechanism so losses are not borne only by late redeemers. One option would be to scale `previewRedeem()` / `previewWithdraw()` outputs based on a real-time reserve ratio (for example comparing the `Treasury.reserve()` to the backed supply), so all redeemers share losses consistently when reserves fall below backing

HV-8 Boolean Toggle Functions Can Counteract Each Other

Informational · Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `7f0a33d`.

File(s) affected: `src/Gateway.sol`, `src/Treasury.sol`

Description: In the `Gateway.toggleWithdrawalDelay()` function and the `Treasury.toggleDepositActive()` / `Treasury.toggleWithdrawActive()` functions, the state is changed by flipping the existing boolean (setting it to `!current`).

This toggle-style pattern does not specify the caller's intended final state, which can cause role-holders to unintentionally counteract each other. If two authorized operators submit these toggle transactions around the same time (or if an operator retries after a pending transaction), the second toggle can revert the system back to the original state. This can lead to operational mistakes such as unintentionally re-enabling deposits/withdrawals or re-enabling the withdrawal delay when the intent was to turn it off.

Recommendation: Replace toggle-style setters with explicit state-setting functions that accept the intended boolean value. For example, replace `Gateway.toggleWithdrawalDelay()` with `Gateway.setWithdrawalDelayEnabled(bool enabled)`

Auditor Suggestions

S1
Single Oracle Failure Causes Reserve Calculation and Harvesting to Revert

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `df0ca52`.
The client provided the following explanation:

> Changed event

File(s) affected: `src/Treasury.sol`

Description: The `Treasury.reserve()` function computes total reserves by iterating over all whitelisted collateral tokens and fetching each token's oracle price to convert balances into `PeggedToken`-denominated units.

However, because `reserve()` relies on `_getPrice()` for each whitelisted token, a failure for any single token oracle will revert the entire `reserve()` calculation. This causes `_excessTokens()` to revert and blocks `Treasury.harvest()`, preventing excess reserve from being

harvested until the oracle issue is resolved.

Recommendation: Consider one of the following approaches depending on the desired resilience:

- **Add a fallback oracle path:** Extend `tokenConfig` to support a secondary oracle (or alternative pricing source) and modify `_getPrice()` / `getPrice()` to fall back when the primary oracle is stale or out of tolerance.
- **Improve revert transparency:** If reverting is intended, add a clearer, dedicated error that includes the failing token address (and optionally the oracle address / failure reason) so operators can immediately identify which oracle failure is blocking `reserve()` and `harvest()`.

S2 Auditor Suggestions and Best Practices

Fixed

Update

Marked as "Fixed" by the client.

Addressed in: `e6d04e6`.

The client provided the following explanation:

> Fixed most of this. Owner address(0) is checked in Ownable contract hence not require to check in this contract.

File(s) affected: `All`

Description: The following best practice suggestions have been identified:

1. `PeggedToken.sol`: Consider inheriting from `IPeggedToken.sol` to ensure interface compatibility.
2. `IPeggedToken.sol`: Consider adding missing function signatures:
    1. `addToBlacklist()`.
    2. `removeFromBlacklist()`.
    3. `updateGateway()`.
    4. `updateTreasury()`.
    5. `getBlacklistedAddresses()`.
    6. `isBlacklisted()`.
3. `IYieldDistributor.sol`: Consider adding missing function signatures:
    1. `asset()`.
    2. `vault()`.
    3. `distribute()`.
    4. `rescueTokens()`.
    5. `updateYieldDuration()`.
4. `Treasury.sol`: Consider inheriting from `ITreasury.sol` to ensure interface compatibility.
5. `ITreasury.sol`: Consider adding missing function signatures:
    1. `addToWhitelist()`.
    2. `removeFromWhitelist()`.
    3. `migrate()`.
    4. `sweep()`.
    5. `updateOracle()`.
    6. `updatePriceTolerance()`.
    7. `updateSweeper()`.
    8. `push()`.
    9. `pull()`.
    10. `toggleDepositActive()`.
    11. `toggleWithdrawActive()`.
    12. `swap()`.
    13. `harvest()`.
    14. `whitelistedTokens()`.
6. Consider keeping the expression in the code, rather than using the precomputed constant. This improves auditability and has no downside, since the expression anyway is computed during compile time and hence does not add any gas:
    1. `YieldDistributor.sol#L44`.
    2. `StakingVault.sol#L51`.
    3. `Gateway.sol#L55`.
7. `Treasury.sol`: Event `WithdrawnAll` remains unused and can therefore be removed.
8. `Gateway.sol`: Errors `ExceededExcessReserve`, `NoExcessReserve` and `TokenNotSupported` remain unused.
9. `Gateway.initialize()`: Parameters `mintLimit_` and `initialWithdrawalDelay_` remain unbound. Consider adding sane upper bounds.
10. `Treasury.sol#L422-L424`: Use of explicit unchecked "i++" inside loops in order to save gas no longer is necessary beginning with solidity `0.8.22`, due to a compiler improvement that automatically lowers these variables accordingly.
11. `Gateway.requestRedeem()`: Consider changing the function name to a more fitting one like `lockupPeggedTokens()` for improved readability.
12. `StakingVault.sol`: Error `RequestNotActive` remains unused and may be removed.
13. `StakingVault._createRequest()`: Missing NatSpec for parameter `assets_`
14. `IStakingVault.sol`: Missing the following functions:
    1. `updateYieldDistributor()`.

2. `updateVaultRewards()`.
3. `updateInstantWithdrawWhitelist()`.
4. `updateCooldownEnabled()`.
5. `updateCooldownDuration()`.

Recommendation: Consider implementing the suggested improvements.

S3 No Explicit Validation for Negative or Zero Oracle Prices

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `7a56ed5`.

File(s) affected: `src/Treasury.sol`

Description: The `_getPrice()` function casts `int256 _price` from the Chainlink oracle to `uint256` without explicitly validating that the price is positive. Even if it's very unlikely (oracle malfunctions, errors, etc.), a negative price from a Chainlink oracle would wrap to a very large `uint256` value and be caught by the tolerance check (exceeding `_priceUpperBound`). A zero price would pass if `priceTolerance` is set to `MAX_BPS` (100%), since `_priceLowerBound` would be 0.

Recommendation: Add explicit price validation before casting:

```
if (_price <= 0) revert InvalidOraclePrice();
_latestPrice = uint256(_price);
```

S4
Unbounded Loops over Dynamic Collections May Cause Gas-Related Dos

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `1286d4b`.

File(s) affected: `src/Treasury.sol`, `src/StakingVault.sol`

Description: Multiple functions in Treasury (`reserve()`, `migrate()`, `sweep()`, `swap()`) loop over the `_whitelistedTokens` set, and functions in StakingVault (`getClaimableRequests()`, `getPendingRequests()`, `claimWithdrawBatch()`) loop over a user's `activeRequestIds` set. Neither collection has an upper bound enforced on-chain.

The most critical case is `Treasury.reserve()` — each iteration performs external calls to a Chainlink oracle and an ERC-4626 vault, so gas cost scales steeply. While the expected number of whitelisted tokens is small (2–3), there is no on-chain cap. If the set grows too large, functions like `reserve()`, `harvest()`, and `migrate()` could exceed the block gas limit and revert.

Recommendation: Consider enforcing a maximum size for the whitelisted tokens set in `addToWhitelist()` (e.g., `MAX_WHITELISTED_TOKENS = 10`). For StakingVault view functions, consider adding pagination parameters (offset + limit).

S5
AMO Burn Requires `msg.sender` to Hold the Tokens, but AMO Mint Sends to an Arbitrary Receiver

Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> AMO is trusted role . AMO won't mint for address that can not return to AMO address.

File(s) affected: `src/Gateway.sol`

Description: `mintToAMO()` mints pegged tokens to an arbitrary `receiver_` address, but `burnFromAMO()` always burns from `msg.sender`. This means the `UMM_ROLE` caller must hold the tokens in order to burn them. If tokens were minted to a different receiver (e.g., a DeFi protocol, liquidity pool, or multisig), that receiver must transfer the tokens back to the `UMM_ROLE` address before `burnFromAMO()` can be

called. If the receiver is a contract that cannot or will not return the tokens, the AMO supply cannot be reduced through `burnFromAMO()`, leaving `amoSupply` permanently inflated and reducing the mintable capacity via `maxMint()`.

```
// mintToAMO: mints to arbitrary receiver
$.peggedToken.mint(receiver_, amount_);

// burnFromAMO: always burns from msg.sender
$.peggedToken.burnFrom(msg.sender, amount_);
```

Recommendation: This is expected behavior for AMO operations, but operators should be aware that tokens minted to external contracts or addresses may not be recoverable for burning. Consider documenting this constraint clearly, or adding an optional `from_` parameter to `burnFromAMO()` that allows burning from a pre-approved address.

S6 Missing Input Validation

Mitigated

> **Update**
>
> During the fix review process we identified that the there were no additional input validation for function `Gateway.updateMintLimit()`.

> **Update**
>
> Marked as "Fixed" by the client.
> Addressed in: `4e69077`.

File(s) affected: `src/Treasury.sol`, `src/PeggedToken.sol`, `src/Gateway.sol`

Description: The protocol includes several privileged configuration and administration entrypoints that set critical addresses and system-wide limits. These functions would benefit from additional input validation:

1. `PeggedToken.constructor`: `owner_` is not validated to be non-zero. Deploying with `owner_ == address(0)` can permanently lock access to `onlyOwner` functions.
2. `Treasury.swap()`: the function does not validate that `swapper` has been initialized. If `swapper` is unset, calls will revert or behave unexpectedly when attempting to approve and execute swaps.
3. `Treasury.migrate()`: the function does not validate that the `PeggedToken` has been updated to point to `newTreasury_`. Migrating funds while `PeggedToken.treasury()` still points to the old treasury can leave the system using stale accounting/roles and may strand reserves in the new treasury while the rest of the system continues referencing the old one.
4. `Gateway.updateMintLimit()`: the function does not validate the new mint limit against current outstanding user-minted supply (`peggedToken.totalSupply() - amoSupply`). Setting `mintLimit` below the existing user supply can effectively freeze future minting and break the intended interpretation of `maxMint()`.

Recommendation: Add the missing input validation checks in each of the locations above.

S7

Access Control Resolution Is Gas-Inefficient Due to Chained External Reads of Role Identifiers

Fixed

> **Update**
>
> Marked as "Fixed" by the client.
> Addressed in: `9040436`.

File(s) affected: `src/Gateway.sol`

Description: The Gateway performs repeated chained calls to retrieve role identifiers from Treasury, e.g.: `Gateway._maintainerRole()` → `ITreasury(treasury()).MAINTAINER_ROLE()` `Gateway._ummRole()` → `ITreasury(treasury()).UMM_ROLE()` `Gateway._defaultAdminRole()` → `ITreasury(treasury()).DEFAULT_ADMIN_ROLE()`

Where `Gateway.treasury()` in turn is defined as `_peggedToken().treasury()`.

These are external calls (and in the current design, also depend on `treasury()` which itself calls into `PeggedToken`) just to fetch what are effectively constants. This increases gas cost and adds avoidable runtime coupling.

Recommendation: We recommend to inline role IDs as `bytes32` constants in `Gateway`, matching Treasury's definitions.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/hemilabs/vetro-contracts`

- `559...374 ./src/Gateway.sol`
- `dcf...9a5 ./src/PeggedToken.sol`
- `346...041 ./src/StakingVault.sol`
- `06e...dc6 ./src/Treasury.sol`
- `0cf...bca ./src/YieldDistributor.sol`
- `0d5...28d ./src/deploy/ProxyImports.sol`
- `f0c...3e5 ./src/interfaces/IGateway.sol`
- `1db...112 ./src/interfaces/IPeggedToken.sol`
- `425...66a ./src/interfaces/IStakingVault.sol`
- `425...b90 ./src/interfaces/ITreasury.sol`
- `4e9...432 ./src/interfaces/IVaultRewards.sol`
- `9fc...fab ./src/interfaces/IYieldDistributor.sol`
- `229...0ab ./src/interfaces/bloq/ISwapper.sol`
- `683...be1 ./src/interfaces/chainlink/IAggregatorV3.sol`

Test Suite Results

forge test

```
No files changed, compilation skipped

Ran 72 tests for test/Treasury.t.sol:TreasuryTest
[PASS] test_addToWhitelist_revertIfAlreadyWhitelisted() (gas: 28698)
[PASS] test_addToWhitelist_revertIfNotDefaultAdmin() (gas: 1257591)
[PASS] test_addToWhitelist_revertOnAssetMismatch() (gas: 533616)
[PASS] test_addToWhitelist_revertOnInvalidTokenDecimals() (gas: 531456)
[PASS] test_addToWhitelist_revertOnStalePeriodZero() (gas: 17759)
[PASS] test_addToWhitelist_revertOnZeroAddress() (gas: 15611)
[PASS] test_addToWhitelist_success() (gas: 1421746)
[PASS] test_deposit_onlyGateway_revertIfDepositPaused() (gas: 32540)
```

```
[PASS] test_deposit_onlyGateway_revertIfNotWhitelisted() (gas: 533532)
[PASS] test_deposit_onlyGateway_success() (gas: 242120)
[PASS] test_deposit_revertIfCallerIsNotGateway() (gas: 16920)
[PASS] test_gateway_returnsPeggedTokenGateway() (gas: 16185)
[PASS] test_getPrice_revertIfOutOfTolerance() (gas: 32505)
[PASS] test_getPrice_revertIfStale() (gas: 25362)
[PASS] test_getPrice_revertIfUnsupportedToken() (gas: 525044)
[PASS] test_getPrice_success() (gas: 26854)
[PASS] test_harvest() (gas: 321739)
[PASS] test_harvest_noExcess() (gas: 292040)
[PASS] test_harvest_revertIfNotUMM() (gas: 17220)
[PASS] test_harvest_revertIfNotWhitelisted() (gas: 560999)
[PASS] test_harvest_revertIfReceiverIsZero() (gas: 48916)
[PASS] test_harvest_withPriceChange() (gas: 300332)
[PASS] test_harvest_withVaultBalance() (gas: 380365)
[PASS] test_isWhitelistedToken() (gas: 13043)
[PASS] test_migrate_revertIfNotDefaultAdmin() (gas: 3271771)
[PASS] test_migrate_revertOnPeggedTokenMismatch() (gas: 4620978)
[PASS] test_migrate_revertOnZeroAddress() (gas: 11044)
[PASS] test_migrate_success() (gas: 3559088)
[PASS] test_onlyKeeper_revertIfCallerIsNotKeeper() (gas: 18384)
[PASS] test_owner_returnsPeggedTokenOwner() (gas: 12948)
[PASS] test_pull_onlyKeeper_revertIfNotWhitelisted() (gas: 560610)
[PASS] test_pull_onlyKeeper_success() (gas: 285216)
[PASS] test_push_onlyKeeper_maxAmountDepositsAllBalance() (gas: 262057)
[PASS] test_push_onlyKeeper_revertIfNotWhitelisted() (gas: 560634)
[PASS] test_push_onlyKeeper_success() (gas: 261244)
[PASS] test_removeFromWhitelist_revertIfBalanceIsNotZero() (gas: 255011)
[PASS] test_removeFromWhitelist_revertIfNotDefaultAdmin() (gas: 18256)
[PASS] test_removeFromWhitelist_revertIfNotWhitelisted() (gas: 527183)
[PASS] test_removeFromWhitelist_success() (gas: 49914)
[PASS] test_reserve_isZero_whenNoTokens() (gas: 28349)
[PASS] test_reserve_tokenInTreasury() (gas: 204744)
[PASS] test_reserve_tokenInVault() (gas: 258716)
[PASS] test_reserve_withMultipleTokens() (gas: 1724079)
[PASS] test_reserve_withPriceChange() (gas: 213109)
[PASS] test_swap_onlyKeeper_revertIfReservedToken() (gas: 48664)
[PASS] test_swap_onlyKeeper_revertOnVaultShareToken() (gas: 57522)
[PASS] test_swap_onlyKeeper_success() (gas: 1140057)
[PASS] test_sweep_revertIfNotDefaultAdmin() (gas: 530309)
[PASS] test_sweep_revertOnReservedToken() (gas: 17724)
[PASS] test_sweep_revertOnVaultShareToken() (gas: 24515)
[PASS] test_sweep_revertOnZeroReceiver() (gas: 525076)
[PASS] test_sweep_success() (gas: 682484)
[PASS] test_toggleDepositActive_onlyKeeper_revertIfNotWhitelisted() (gas: 560490)
[PASS] test_toggleDepositActive_onlyKeeper_success() (gas: 65516)
[PASS] test_toggleWithdrawActive_onlyKeeper_revertIfNotWhitelisted() (gas: 560423)
[PASS] test_toggleWithdrawActive_onlyKeeper_success() (gas: 65594)
[PASS] test_updateOracle_revertIfNotWhitelisted() (gas: 651910)
[PASS] test_updateOracle_revertOnStalePeriodZero() (gas: 139918)
[PASS] test_updateOracle_revertOnZeroAddress() (gas: 18163)
[PASS] test_updateOracle_success() (gas: 158466)
[PASS] test_updatePriceTolerance_revertIfTooHigh() (gas: 14359)
[PASS] test_updatePriceTolerance_success() (gas: 20620)
[PASS] test_updateSwapper_revertOnZeroAddress() (gas: 13523)
[PASS] test_updateSwapper_success() (gas: 40007)
[PASS] test_whitelistedTokens() (gas: 13953)
[PASS] test_withdraw_onlyGateway_revertIfNotWhitelisted() (gas: 535646)
[PASS] test_withdraw_onlyGateway_revertIfWithdrawPaused() (gas: 41313)
[PASS] test_withdraw_onlyGateway_withdrawFromTreasury() (gas: 222030)
[PASS] test_withdraw_onlyGateway_withdrawFromTreasuryAndVault() (gas: 276028)
[PASS] test_withdraw_onlyGateway_withdrawFromVault() (gas: 272923)
[PASS] test_withdrawable() (gas: 261878)
[PASS] test_withdrawable_returnZeroIfNotWhitelisted() (gas: 524757)
Suite result: ok. 72 passed; 0 failed; 0 skipped; finished in 8.16ms (7.12ms CPU time)

Ran 35 tests for test/YieldDistributor.t.sol:YieldDistributorTest
[PASS] testFuzz_distribute(uint256) (runs: 256, μ: 224553, ~: 224736)
[PASS] testFuzz_distributeAfterPeriodEnds(uint256,uint256,uint256) (runs: 256, μ: 265909, ~: 266069)
[PASS] testFuzz_multipleDistributions(uint256,uint256,uint256) (runs: 256, μ: 234747, ~: 234916)
[PASS] testFuzz_multiplePulls(uint256,uint8) (runs: 256, μ: 269595, ~: 268780)
[PASS] testFuzz_pendingYield_linearDistribution(uint256,uint256) (runs: 256, μ: 222818, ~: 222791)
```

```
[PASS] testFuzz_pullYield(uint256,uint256) (runs: 256, μ: 252096, ~: 252754)
[PASS] testFuzz_pullYield_fullPeriod(uint256,uint256) (runs: 256, μ: 248257, ~: 248401)
[PASS] testFuzz_yieldDuration(uint256,uint256) (runs: 256, μ: 232839, ~: 232914)
[PASS] test_distributeAfterPeriodEnds() (gas: 147445)
[PASS] test_distribute_combinesWithRemaining() (gas: 147857)
[PASS] test_distribute_revert_notDistributor() (gas: 18404)
[PASS] test_distribute_revert_zeroAmount() (gas: 18378)
[PASS] test_distribute_success() (gas: 141797)
[PASS] test_grantDistributorRole() (gas: 49914)
[PASS] test_initialize_revert_alreadyInitialized() (gas: 22532)
[PASS] test_initialize_revert_zeroAsset() (gas: 1438316)
[PASS] test_initialize_revert_zeroOwner() (gas: 1438369)
[PASS] test_initialize_revert_zeroVault() (gas: 1438330)
[PASS] test_initialize_success() (gas: 34732)
[PASS] test_multiplePullsWithNoTimePassed() (gas: 172698)
[PASS] test_pendingYield_calculation() (gas: 141584)
[PASS] test_pendingYield_capsAtPeriodFinish() (gas: 138466)
[PASS] test_pendingYield_zeroBeforeDistribution() (gas: 15693)
[PASS] test_pullYieldAfterPeriodEnds() (gas: 172979)
[PASS] test_pullYield_multipleTimesInPeriod() (gas: 187031)
[PASS] test_pullYield_revert_notVault() (gas: 17965)
[PASS] test_pullYield_success() (gas: 176363)
[PASS] test_pullYield_zeroWhenNothingDistributed() (gas: 20240)
[PASS] test_revokeDistributorRole() (gas: 28656)
[PASS] test_updateYieldDuration_affectsFutureDistributions() (gas: 147331)
[PASS] test_updateYieldDuration_revert_belowMinimum() (gas: 18701)
[PASS] test_updateYieldDuration_revert_notOwner() (gas: 18242)
[PASS] test_updateYieldDuration_success() (gas: 25972)
[PASS] test_vaultPullsYieldOnDeposit() (gas: 420122)
[PASS] test_vaultPullsYieldOnRequestRedeem() (gas: 544052)
Suite result: ok. 35 passed; 0 failed; 0 skipped; finished in 331.35ms (330.25ms CPU time)

Ran 52 tests for test/StakingVault.t.sol:StakingVaultTest
[PASS] testFuzz_cancelWithdraw(uint256,uint256) (runs: 256, μ: 375393, ~: 375392)
[PASS] testFuzz_claimWithdraw(uint256,uint256,uint256) (runs: 256, μ: 371032, ~: 370969)
[PASS] testFuzz_cooldownDuration(uint256,uint256) (runs: 256, μ: 365468, ~: 365550)
[PASS] testFuzz_deposit(uint256) (runs: 256, μ: 211588, ~: 211746)
[PASS] testFuzz_instantWithdraw_whenWhitelisted(uint256,uint256) (runs: 256, μ: 260220, ~: 260089)
[PASS] testFuzz_mint(uint256) (runs: 256, μ: 209340, ~: 209542)
[PASS] testFuzz_multipleRequests(uint256,uint8) (runs: 256, μ: 742250, ~: 603105)
[PASS] testFuzz_requestRedeem(uint256,uint256) (runs: 256, μ: 363418, ~: 363276)
[PASS] testFuzz_requestWithdraw(uint256,uint256) (runs: 256, μ: 360959, ~: 360813)
[PASS] testFuzz_shareValue_afterYield(uint256,uint256) (runs: 256, μ: 633520, ~: 633349)
[PASS] test_cancelWithdraw_revert_alreadyClaimed() (gas: 270096)
[PASS] test_cancelWithdraw_revert_notOwner() (gas: 340957)
[PASS] test_cancelWithdraw_success() (gas: 273917)
[PASS] test_claimWithdrawBatch_success() (gas: 392985)
[PASS] test_claimWithdraw_revert_alreadyClaimed() (gas: 270355)
[PASS] test_claimWithdraw_revert_invalidRequestId() (gas: 23530)
[PASS] test_claimWithdraw_revert_notMatured() (gas: 309465)
[PASS] test_claimWithdraw_revert_notOwner() (gas: 337644)
[PASS] test_claimWithdraw_revert_zeroReceiver() (gas: 306585)
[PASS] test_claimWithdraw_success() (gas: 271208)
[PASS] test_claimWithdraw_toReceiver() (gas: 273550)
[PASS] test_deposit_success() (gas: 125481)
[PASS] test_getActiveRequestIds() (gas: 431624)
[PASS] test_getClaimableRequests() (gas: 439828)
[PASS] test_getClaimableRequests_arrayLengthReduction() (gas: 615177)
[PASS] test_getPendingRequests() (gas: 436762)
[PASS] test_globalRequestIds_unique() (gas: 509763)
[PASS] test_initialize_revert_alreadyInitialized() (gas: 21251)
[PASS] test_initialize_revert_zeroAsset() (gas: 2705986)
[PASS] test_initialize_revert_zeroOwner() (gas: 2706048)
[PASS] test_initialize_success() (gas: 42006)
[PASS] test_mint_success() (gas: 122605)
[PASS] test_redeem_revert_whenCooldownEnabled() (gas: 127962)
[PASS] test_requestRedeem_revert_insufficientShares() (gas: 59044)
[PASS] test_requestRedeem_revert_whenCooldownDisabled() (gas: 37463)
[PASS] test_requestRedeem_revert_zeroAssets() (gas: 29489)
[PASS] test_requestRedeem_success() (gas: 312346)
[PASS] test_requestRedeem_withAllowance() (gas: 318475)
[PASS] test_requestWithdraw_revert_zeroAmount() (gas: 29556)
```

```
[PASS] test_requestWithdraw_success() (gas: 306649)
[PASS] test_totalAssets_excludesCooldown() (gas: 311845)
[PASS] test_updateCooldownDuration() (gas: 25869)
[PASS] test_updateCooldownDuration_revert_aboveMax() (gas: 18613)
[PASS] test_updateCooldownDuration_revert_belowMin() (gas: 18537)
[PASS] test_updateCooldownEnabled() (gas: 21225)
[PASS] test_updateInstantWithdrawWhitelist() (gas: 45836)
[PASS] test_updateInstantWithdrawWhitelist_revert_zeroAddress() (gas: 18372)
[PASS] test_updateYieldDistributor() (gas: 27927)
[PASS] test_withdraw_revert_whenCooldownEnabled() (gas: 127873)
[PASS] test_withdraw_success_whenCooldownDisabled() (gas: 159634)
[PASS] test_withdraw_success_whenWhitelisted() (gas: 186256)
[PASS] test_yieldDistribution_increasesShareValue() (gas: 459844)
Suite result: ok. 52 passed; 0 failed; 0 skipped; finished in 585.57ms (586.12ms CPU time)

Ran 81 tests for test/Gateway.t.sol:GatewayTest
[PASS] testFuzz_deposit(int256,uint256,uint256) (runs: 256, μ: 406552, ~: 407848)
[PASS] testFuzz_mint(int256,uint256,uint256) (runs: 256, μ: 410586, ~: 411684)
[PASS] testFuzz_previewDeposit(int256,uint256,uint256) (runs: 256, μ: 108454, ~: 108681)
[PASS] testFuzz_previewMint(int256,uint256,uint256) (runs: 256, μ: 107242, ~: 107504)
[PASS] testFuzz_previewRedeem(int256,uint256,uint256) (runs: 256, μ: 91885, ~: 92127)
[PASS] testFuzz_previewWithdraw(int256,uint256,uint256) (runs: 256, μ: 92165, ~: 92347)
[PASS] testFuzz_redeem(int256,uint256,uint256) (runs: 256, μ: 447187, ~: 438614)
[PASS] testFuzz_withdraw(int256,uint256,uint256) (runs: 256, μ: 458379, ~: 459674)
[PASS] test_addToInstantRedeemWhitelist() (gas: 108793)
[PASS] test_addToInstantRedeemWhitelist_revertIfAlreadyWhitelisted() (gas: 104904)
[PASS] test_addToInstantRedeemWhitelist_revertIfNotMaintainerRole() (gas: 36516)
[PASS] test_addToInstantRedeemWhitelist_revertIfZeroAddress() (gas: 28551)
[PASS] test_burnFromAMO_onlyUMMRole_burnAll() (gas: 116188)
[PASS] test_burnFromAMO_onlyUMMRole_revertIfExceedsAmoSupply() (gas: 179972)
[PASS] test_burnFromAMO_onlyUMMRole_revertIfNotAuthorized() (gas: 34123)
[PASS] test_burnFromAMO_onlyUMMRole_revertIfZeroAmoSupply() (gas: 89655)
[PASS] test_burnFromAMO_onlyUMMRole_success() (gas: 157087)
[PASS] test_cancelRedeemRequest() (gas: 484536)
[PASS] test_cancelRedeemRequest_revertIfNoRequest() (gas: 20516)
[PASS] test_deposit_revertIfMaxMintExceeded() (gas: 256252)
[PASS] test_deposit_revertIfMintableIsNotEnough() (gas: 243242)
[PASS] test_deposit_revertIfTokenHasFeeOnTransfer() (gas: 282903)
[PASS] test_deposit_revertIfTokenIsUnsupported() (gas: 706886)
[PASS] test_getRedeemRequest() (gas: 458412)
[PASS] test_maxDeposit() (gas: 13359)
[PASS] test_maxMint() (gas: 109027)
[PASS] test_maxRedeem() (gas: 78972)
[PASS] test_maxWithdraw() (gas: 205372)
[PASS] test_mintToAMO_onlyUMMRole() (gas: 292459)
[PASS] test_mintToAMO_onlyUMMRole_amoLimitIndependentOfUserLimit() (gas: 311292)
[PASS] test_mintToAMO_onlyUMMRole_revertIfAmoMintLimitExceeded() (gas: 66748)
[PASS] test_mintToAMO_onlyUMMRole_revertIfNotAuthorized() (gas: 34411)
[PASS] test_mintToAMO_onlyUMMRole_revertIfReceiverIsZeroAddress() (gas: 28517)
[PASS] test_mint_revertIfExcessiveInput() (gas: 243267)
[PASS] test_mint_revertIfMaxMintExceeded() (gas: 260197)
[PASS] test_mint_revertIfTokenIsUnsupported() (gas: 739147)
[PASS] test_owner_and_treasury_views() (gas: 33405)
[PASS] test_previewDeposit_revertForUnsupportedToken() (gas: 28354)
[PASS] test_previewMint_revertForUnsupportedToken() (gas: 541812)
[PASS] test_previewRedeem_revertForUnsupportedToken() (gas: 28399)
[PASS] test_previewWithdraw_revertForUnsupportedToken() (gas: 28927)
[PASS] test_redeem_afterRequest_excessAmount_notWhitelisted_reverts() (gas: 483435)
[PASS] test_redeem_afterRequest_excessAmount_whitelisted() (gas: 582548)
[PASS] test_redeem_afterRequest_fullAmount() (gas: 517972)
[PASS] test_redeem_afterRequest_partialAmount() (gas: 515480)
[PASS] test_redeem_instantWhenDelayDisabled() (gas: 434817)
[PASS] test_redeem_instantWithWhitelist() (gas: 486370)
[PASS] test_redeem_revertIfMaxWithdrawExceeded() (gas: 216024)
[PASS] test_redeem_revertIfNotWhitelisted_delayEnabled() (gas: 382517)
[PASS] test_redeem_revertIfRedeemableIsNotEnough() (gas: 384222)
[PASS] test_redeem_revertIfTokenIsUnsupported() (gas: 31520)
[PASS] test_removeFromInstantRedeemWhitelist() (gas: 87959)
[PASS] test_removeFromInstantRedeemWhitelist_revertIfNotInWhitelist() (gas: 33455)
[PASS] test_removeFromInstantRedeemWhitelist_revertIfNotMaintainerRole() (gas: 111843)
[PASS] test_requestRedeem() (gas: 460823)
[PASS] test_requestRedeem_mergingRequests() (gas: 466824)
```

```
[PASS] test_requestRedeem_revertIfWithdrawalDelayDisabled() (gas: 400516)
[PASS] test_requestRedeem_revertIfZeroAmount() (gas: 16218)
[PASS] test_toggleWithdrawalDelay() (gas: 46789)
[PASS] test_toggleWithdrawalDelay_revertIfNotMaintainerRole() (gas: 34183)
[PASS] test_updateAmoMintLimit_allowZeroWhenAmoSupplyIsZero() (gas: 38275)
[PASS] test_updateAmoMintLimit_revertIfBelowCurrentAmoSupply() (gas: 291094)
[PASS] test_updateAmoMintLimit_revertIfNotDefaultAdminRole() (gas: 31456)
[PASS] test_updateAmoMintLimit_success() (gas: 58174)
[PASS] test_updateMintFee() (gas: 53433)
[PASS] test_updateMintFee_revertIfFeeIsHigherThanMax() (gas: 28820)
[PASS] test_updateMintFee_revertIfNotMaintainerRole() (gas: 34217)
[PASS] test_updateMintLimit() (gas: 38846)
[PASS] test_updateMintLimit_revertIfNotDefaultAdminRole() (gas: 31457)
[PASS] test_updateRedeemFee() (gas: 36551)
[PASS] test_updateRedeemFee_revertIfFeeIsHigherThanMax() (gas: 28830)
[PASS] test_updateRedeemFee_revertIfNotMaintainerRole() (gas: 34382)
[PASS] test_updateWithdrawalDelay() (gas: 38196)
[PASS] test_updateWithdrawalDelay_revertIfNotMaintainerRole() (gas: 34315)
[PASS] test_updateWithdrawalDelay_revertIfZero() (gas: 28607)
[PASS] test_withdraw_afterRequest_fullAmount() (gas: 517348)
[PASS] test_withdraw_afterRequest_partialAmount() (gas: 522674)
[PASS] test_withdraw_revertIfMaxWithdrawExceeded() (gas: 163773)
[PASS] test_withdraw_revertIfNotWhitelisted_delayEnabled() (gas: 391434)
[PASS] test_withdraw_revertIfPeggedExcessiveInput() (gas: 65645)
[PASS] test_withdraw_revertIfTokenIsUnsupported() (gas: 32074)
Suite result: ok. 81 passed; 0 failed; 0 skipped; finished in 655.05ms (671.00ms CPU time)

Ran 3 tests for test/VUSD_E2E.t.sol:VUSD_E2E_Test
[PASS] test_deposit_pushesFundsIntoVault() (gas: 445341)
[PASS] test_mint_excess_after_yield() (gas: 710370)
[PASS] test_redeem_afterStrategyRebalance() (gas: 787420)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 5.19s (2.68s CPU time)

Ran 9 tests for test/invariant/YieldDistributor.invariant.t.sol:YieldDistributorInvariantTest
[PASS] invariant_balanceCoverssPending() (runs: 256, calls: 128000, reverts: 0)
[PASS] invariant_callSummary() (runs: 256, calls: 128000, reverts: 0)
[PASS] invariant_lastUpdateTimeAfterPull() (runs: 256, calls: 128000, reverts: 0)
[PASS] invariant_noPendingWithoutDistribution() (runs: 256, calls: 128000, reverts: 0)
[PASS] invariant_periodFinishConsistency() (runs: 256, calls: 128000, reverts: 0)
[PASS] invariant_pulledNeverExceedsDistributed() (runs: 256, calls: 128000, reverts: 0)
[PASS] invariant_rewardRateConsistency() (runs: 256, calls: 128000, reverts: 0)
[PASS] invariant_yieldAccountingConsistency() (runs: 256, calls: 128000, reverts: 0)
[PASS] invariant_yieldDurationMinimum() (runs: 256, calls: 128000, reverts: 0)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 24.22s (155.52s CPU time)

Ran 27 tests for test/PeggedToken.t.sol:PeggedTokenTest
[PASS] test_addToBlacklist_revertIfAlreadyBlacklisted() (gas: 82710)
[PASS] test_addToBlacklist_revertIfNotOwner() (gas: 15738)
[PASS] test_addToBlacklist_revertIfZeroAddress() (gas: 10727)
[PASS] test_addToBlacklist_success() (gas: 83843)
[PASS] test_burnFrom_byAnotherUser() (gas: 80330)
[PASS] test_burnFrom_byGateway() (gas: 53813)
[PASS] test_burnFrom_revertIfBlacklisted() (gas: 146967)
[PASS] test_getBlacklistedAddresses() (gas: 110549)
[PASS] test_mint() (gas: 67737)
[PASS] test_mint_revertIfNotGateway() (gas: 15759)
[PASS] test_mint_revertIfToBlacklisted() (gas: 87396)
[PASS] test_removeFromBlacklist_revertIfNotBlacklisted() (gas: 15667)
[PASS] test_removeFromBlacklist_revertIfNotOwner() (gas: 84846)
[PASS] test_removeFromBlacklist_success() (gas: 65734)
[PASS] test_transfer() (gas: 98177)
[PASS] test_transferFrom() (gas: 131000)
[PASS] test_transferFrom_revertIfFromBlacklisted() (gas: 166744)
[PASS] test_transferFrom_revertIfToBlacklisted() (gas: 169073)
[PASS] test_transfer_revertIfFromBlacklisted() (gas: 137898)
[PASS] test_transfer_revertIfToBlacklisted() (gas: 140153)
[PASS] test_updateGateway_revertIfNotOwner() (gas: 15760)
[PASS] test_updateGateway_revertIfTreasuryIsNotSet() (gas: 1371387)
[PASS] test_updateGateway_revertIfZeroAddress() (gas: 12841)
[PASS] test_updateGateway_success() (gas: 24489)
[PASS] test_updateTreasury_revertIfNotOwner() (gas: 15782)
[PASS] test_updateTreasury_revertIfZeroAddress() (gas: 10707)
```

```
[PASS] test_updateTreasury_success() (gas: 22371)
Suite result: ok. 27 passed; 0 failed; 0 skipped; finished in 56.93s (1.44ms CPU time)

Ran 8 tests for test/invariant/StakingVault.invariant.t.sol:StakingVaultInvariantTest
[PASS] invariant_activeRequestsHavePositiveAssets() (runs: 256, calls: 128000, reverts: 1871)
[PASS] invariant_assetsAccountingConsistency() (runs: 256, calls: 128000, reverts: 1789)
[PASS] invariant_callSummary() (runs: 256, calls: 128000, reverts: 1915)
[PASS] invariant_cooldownAssetsMatchActiveRequests() (runs: 256, calls: 128000, reverts: 1868)
[PASS] invariant_noSharesImpliesNoStakedAssets() (runs: 256, calls: 128000, reverts: 1854)
[PASS] invariant_requestIdMonotonicallyIncreasing() (runs: 256, calls: 128000, reverts: 1851)
[PASS] invariant_sharePriceNeverDecreases() (runs: 256, calls: 128000, reverts: 1902)
[PASS] invariant_totalAssetsCalculation() (runs: 256, calls: 128000, reverts: 1865)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 56.93s (240.21s CPU time)

Ran 8 test suites in 56.93s (144.84s CPU time): 287 tests passed, 0 failed, 0 skipped (287 total tests)
```

Code Coverage

forge coverage

File	% Lines	% Statements	% Branches	% Funcs
src/Gateway.sol	96.15% (250/260)	95.61% (283/296)	87.10% (27/31)	94.64% (53/56)
src/PeggedToken.sol	100.00% (33/33)	100.00% (37/37)	100.00% (10/10)	100.00% (9/9)
src/StakingVault.sol	93.41% (170/182)	93.91% (185/197)	85.71% (18/21)	94.44% (34/36)
src/Treasury.sol	100.00% (178/178)	99.58% (238/239)	97.78% (44/45)	100.00% (25/25)
src/YieldDistributor.sol	93.55% (58/62)	92.19% (59/64)	80.00% (8/10)	93.33% (14/15)
test/invariant/StakingVault.invariant.t.sol	100.00% (62/62)	100.00% (62/62)	100.00% (0/0)	100.00% (10/10)
test/invariant/YieldDistributor.invariant.t.sol	100.00% (28/28)	100.00% (24/24)	100.00% (0/0)	100.00% (5/5)
test/mocks/MockChainlinkOracle.sol	100.00% (9/9)	100.00% (5/5)	100.00% (0/0)	100.00% (4/4)
test/mocks/MockERC20.sol	100.00% (14/14)	100.00% (13/13)	100.00% (3/3)	100.00% (4/4)
test/mocks/MockSwapper.sol	100.00% (6/6)	100.00% (5/5)	50.00% (2/4)	100.00% (1/1)
test/mocks/MockYieldVault.sol	76.47% (26/34)	76.67% (23/30)	30.00% (3/10)	77.78% (7/9)
Total	96.08% (834/868)	96.09% (934/972)	85.82% (115/134)	95.40% (166/174)

Changelog

- 2026-02-20 - Initial report
- 2026-03-10 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over $200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, PolyMarket, PandcakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.



© 2026 – Quantstamp, Inc.